CODE:-

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import AgglomerativeClustering

mall_data = pd.read_csv("Mall_Customers.csv")
mall_data.head()

#values
mall_data.info()


# We can use a heatmap to check correlation between the variables.
corr = mall_data.select_dtypes(include="number").corr()

plt.figure(figsize=(8, 8))
sns.heatmap(
    corr,
    cbar=True,
    square=True,
    fmt=".1f",
    annot=True,
    cmap="Reds"
)
plt.title("Correlation Heatmap")
plt.show()


# Which gender shops more?
plt.figure(figsize=(10,10))
sns.countplot(x="Gender", data=mall_data)


# People of what ages shop more?
plt.figure(figsize=(16,10))
sns.countplot(x="Age", data=mall_data)

# Is there any relationship between annual income and spending score?
plt.figure(figsize=(20,8))
sns.barplot(x='Annual Income (k$)',y='Spending Score (1-100)',data=mall_data)


# For our model, we can choose those variables which are relevant or necessary, we needn't choose all.
# Choose age, annual income and spending score columns for clustering model.
X = mall_data.iloc[:,[2,3,4]].values
```

```
#Now, we need to find the optimal number of clusters for this dataset.
#To do that, we will use WCSS (within-cluster-sum-of-squares) parameter.
#WCSS is the sum of squares of the distances of each data point in all clusters to their respective
centroids.
#The idea is to minimise the sum.
wcss = []
for i in range(1,11):#It will find wcss value for different number of clusters (for 1 cluster, for 2...until 10
clusters) and put it in list
    kmeans = KMeans(n_clusters=i, init='k-means++', random_state=50)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)
wcss

# elbow graph
sns.set()
plt.plot(range(1,11),wcss)
plt.xlabel("Number of clusters")
plt.ylabel("WCSS value")
plt.show()



#Building the model
kmeans = KMeans(n_clusters = 5, init = 'k-means++',random_state = 0)


# we need a label for each datapoint relative to their clusters
#(will be split into 5 clusters and each will be labelled 0-4)
y = kmeans.fit_predict(X)


# 3d scatterplot using matplotlib

fig = plt.figure(figsize = (10,10))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(X[y == 0,0],X[y == 0,1],X[y == 0,2], s = 40 , color = 'red', label = "cluster 1")
ax.scatter(X[y == 1,0],X[y == 1,1],X[y == 1,2], s = 40 , color = 'blue', label = "cluster 2")
ax.scatter(X[y == 2,0],X[y == 2,1],X[y == 2,2], s = 40 , color = 'green', label = "cluster 3")
ax.scatter(X[y == 3,0],X[y == 3,1],X[y == 3,2], s = 40 , color = 'yellow', label = "cluster 4")
ax.scatter(X[y == 4,0],X[y == 4,1],X[y == 4,2], s = 40 , color = 'purple', label = "cluster 5")
ax.set_xlabel('Age of a customer-->')
ax.set_ylabel('Anual Income-->')
ax.set_zlabel('Spending Score-->')
ax.legend()
plt.show()

# Dendrogram to identify the suitable number of clusters
plt.figure(figsize=(12, 7))
linked = linkage(X, method='ward')
dendrogram(linked)
```

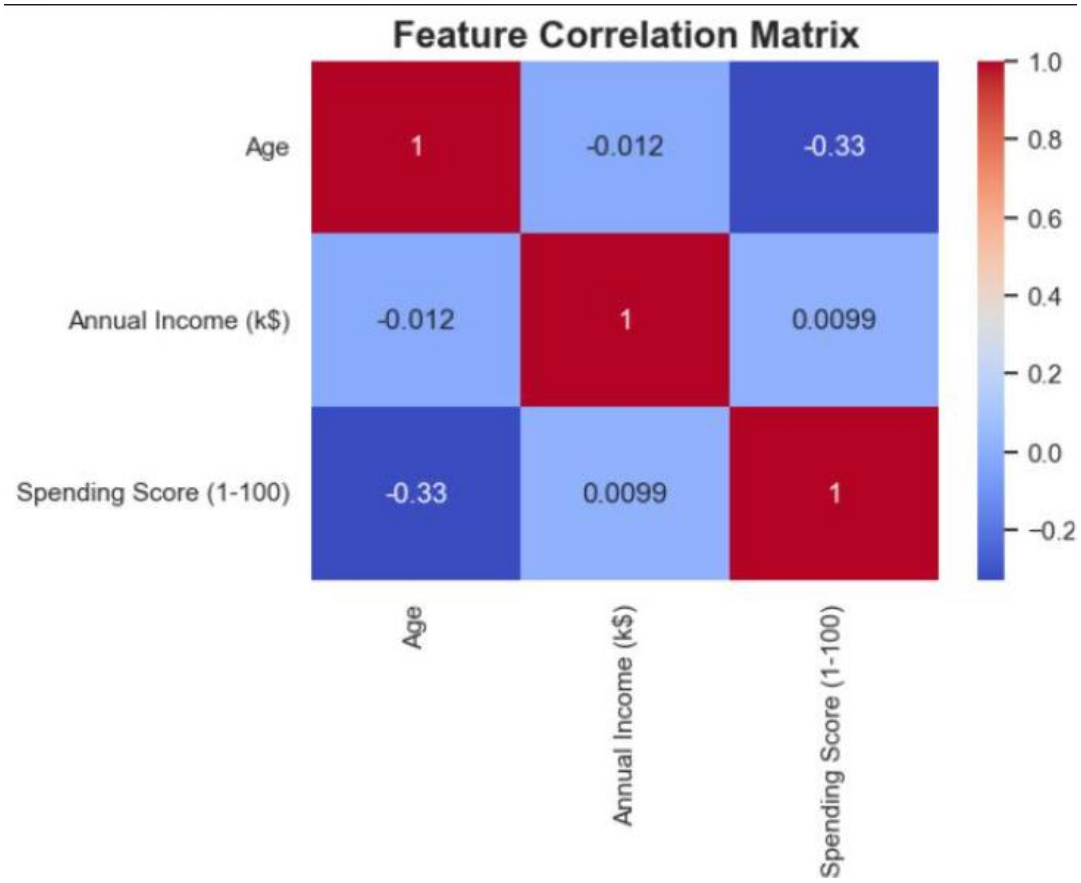
```
plt.title('Dendrogram')
plt.xlabel('Customers')
plt.ylabel('Euclidean Distance')
plt.show()

# Building the Hierarchical Clustering model
hc = AgglomerativeClustering(n_clusters=5, linkage='ward')
y_hc = hc.fit_predict(X)

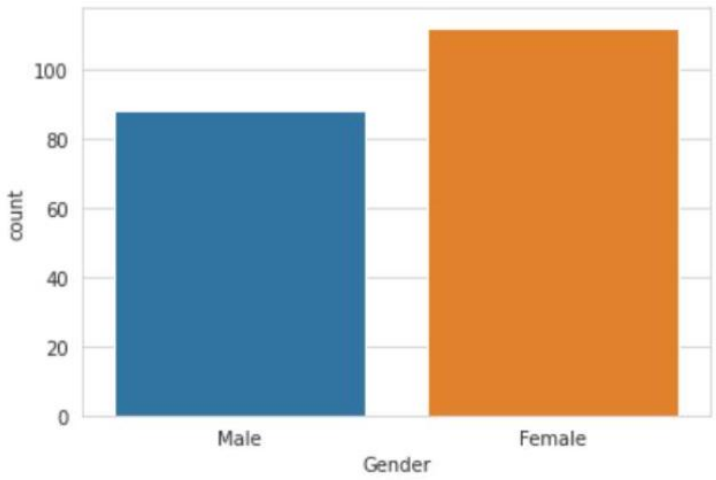
# 3D scatterplot for Hierarchical Clustering
fig = plt.figure(figsize=(10,10))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(X[y_hc == 0,0], X[y_hc == 0,1], X[y_hc == 0,2], s=40, color='red', label='cluster 1')
ax.scatter(X[y_hc == 1,0], X[y_hc == 1,1], X[y_hc == 1,2], s=40, color='blue', label='cluster 2')
ax.scatter(X[y_hc == 2,0], X[y_hc == 2,1], X[y_hc == 2,2], s=40, color='green', label='cluster 3')
ax.scatter(X[y_hc == 3,0], X[y_hc == 3,1], X[y_hc == 3,2], s=40, color='yellow', label='cluster 4')
ax.scatter(X[y_hc == 4,0], X[y_hc == 4,1], X[y_hc == 4,2], s=40, color='purple', label='cluster 5')
ax.set_xlabel('Age of a customer-->')
ax.set_ylabel('Anual Income-->')
ax.set_zlabel('Spending Score-->')
ax.set_title('Hierarchical Clustering')
ax.legend()
plt.show()
```

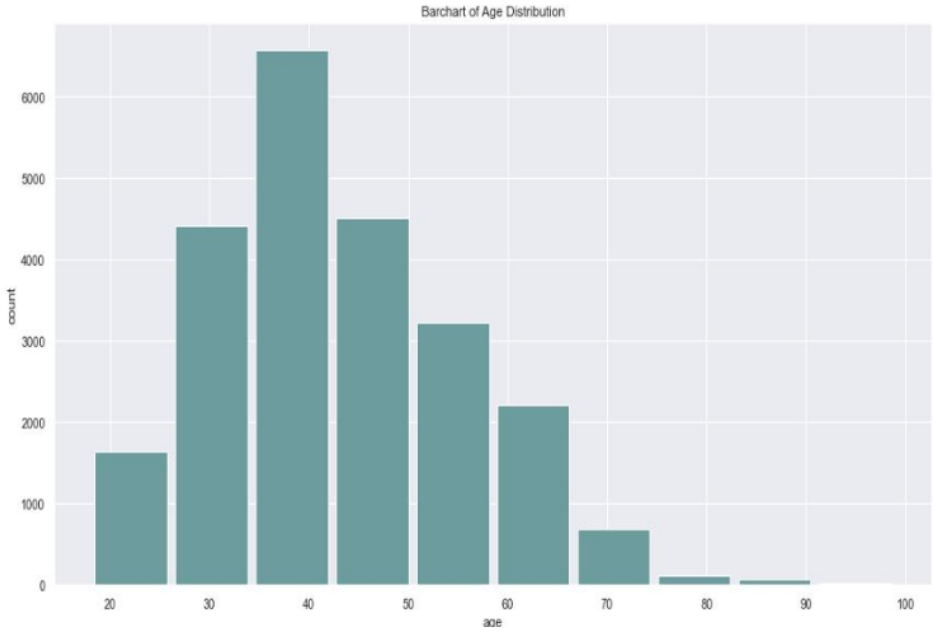
OUTPUT:-

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: approximately 7.9+ KB
```



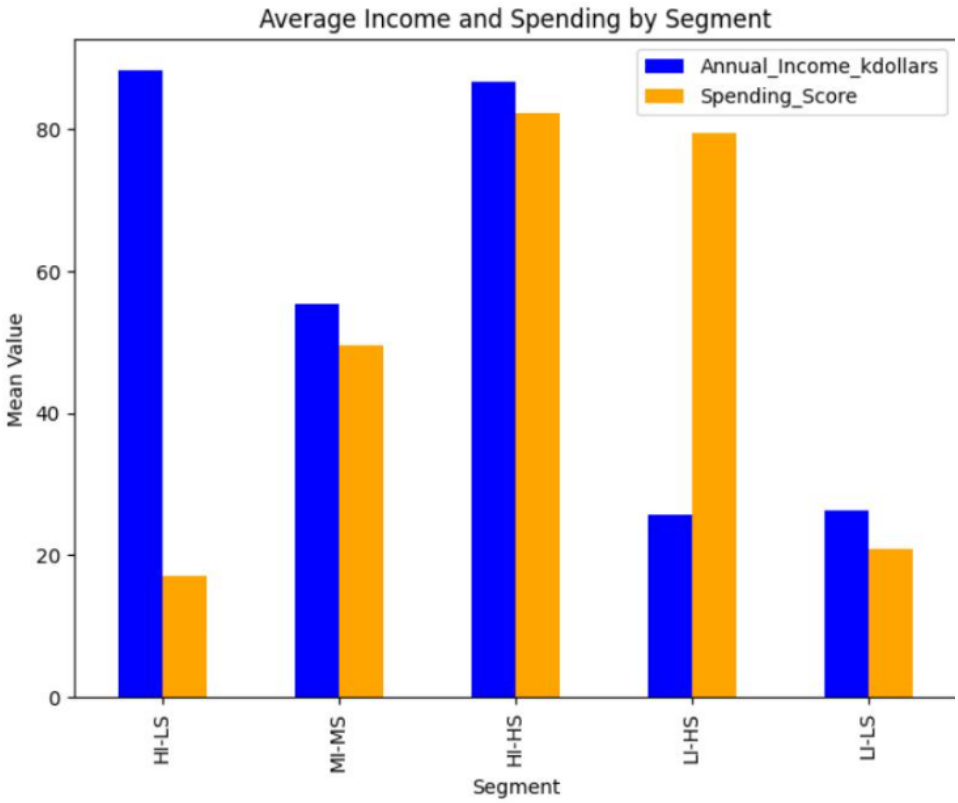
```
Female    112
Male       88
Name: Gender, dtype: int64
```

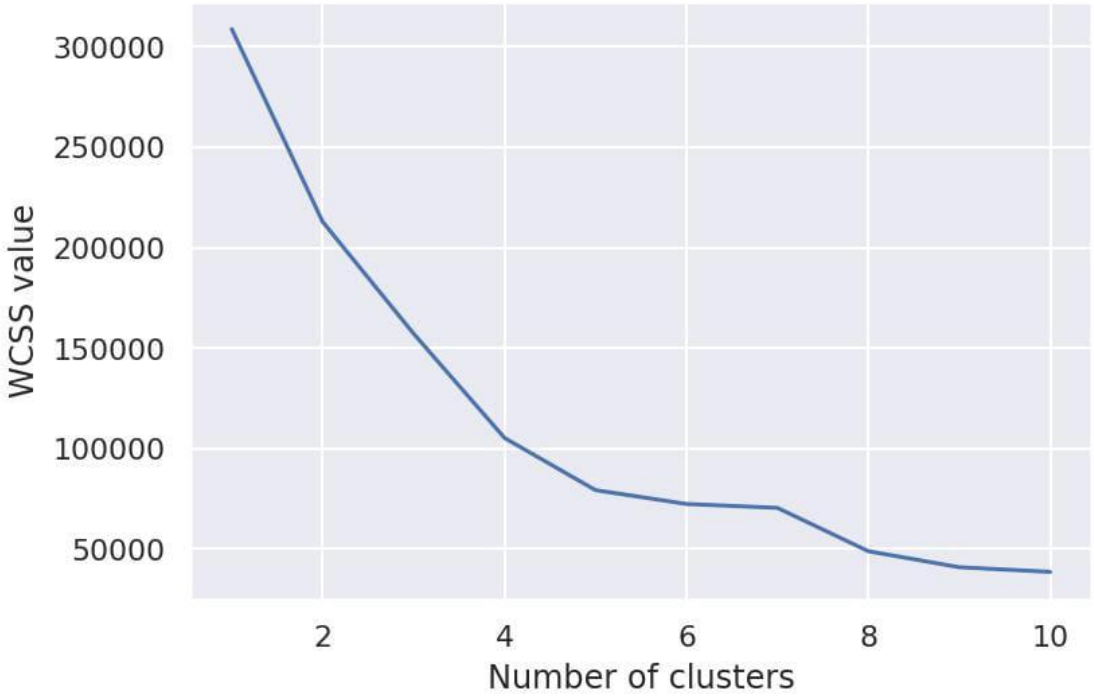


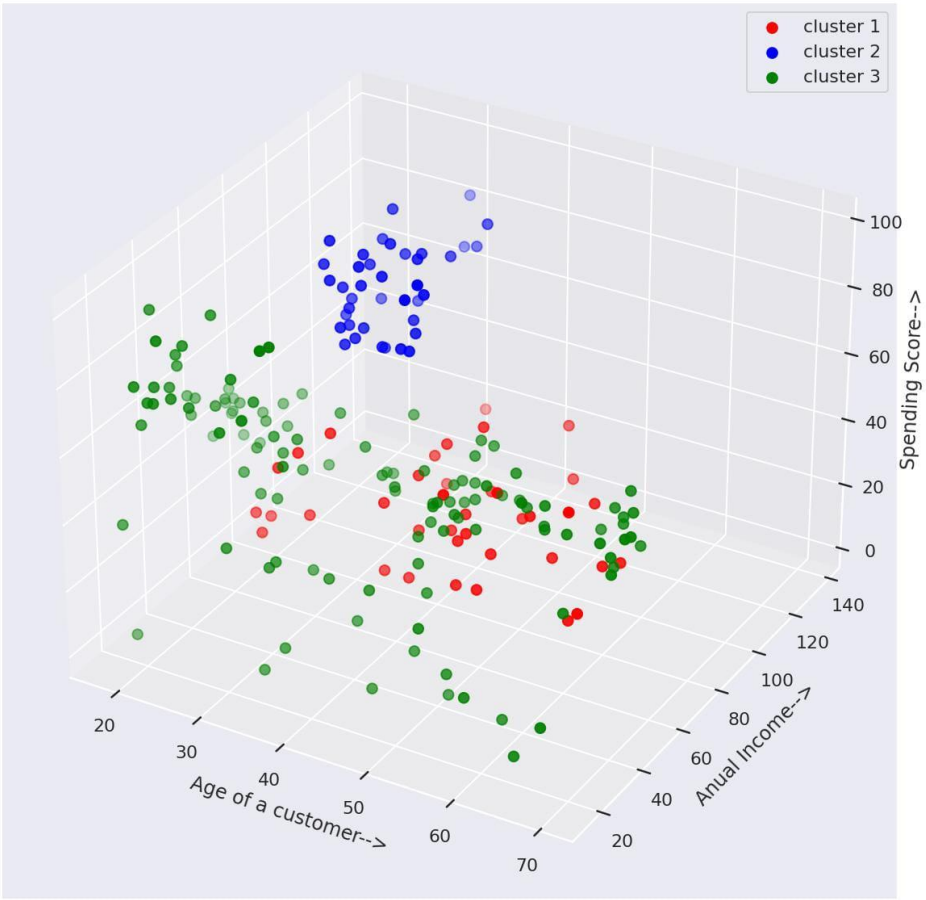


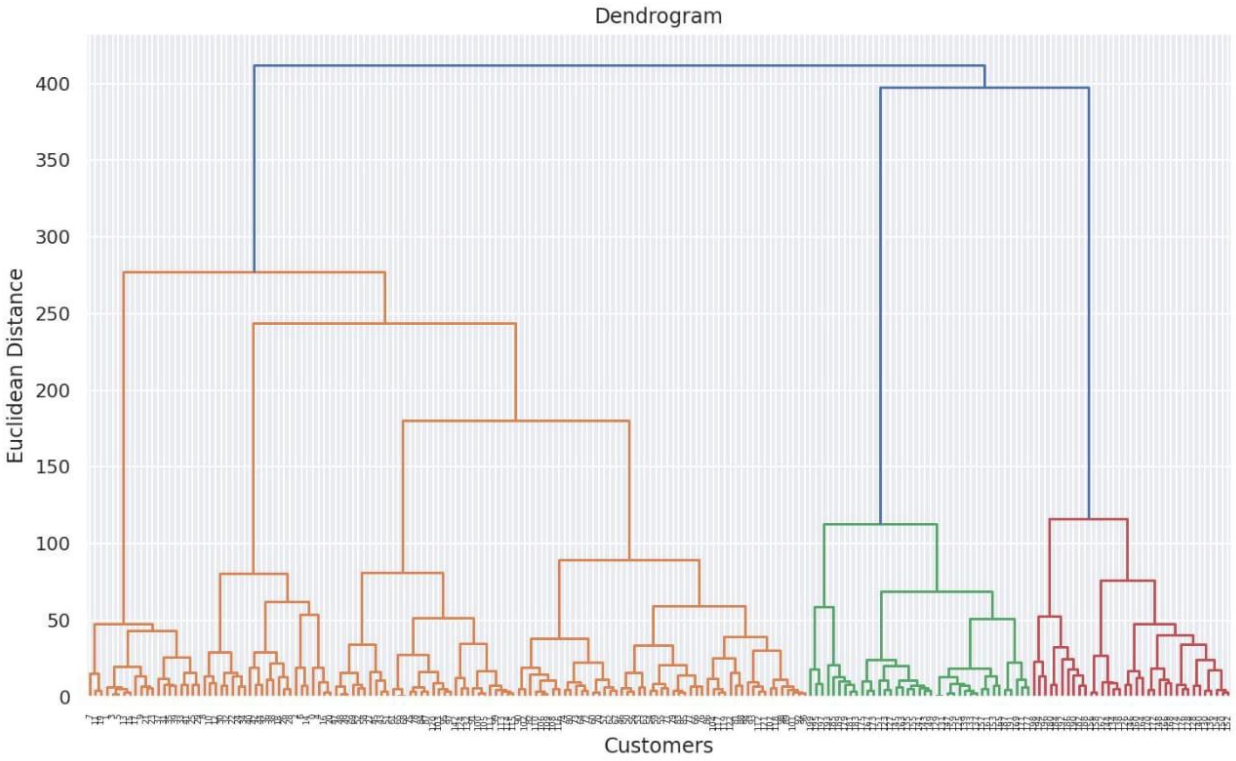
WCSS Values:

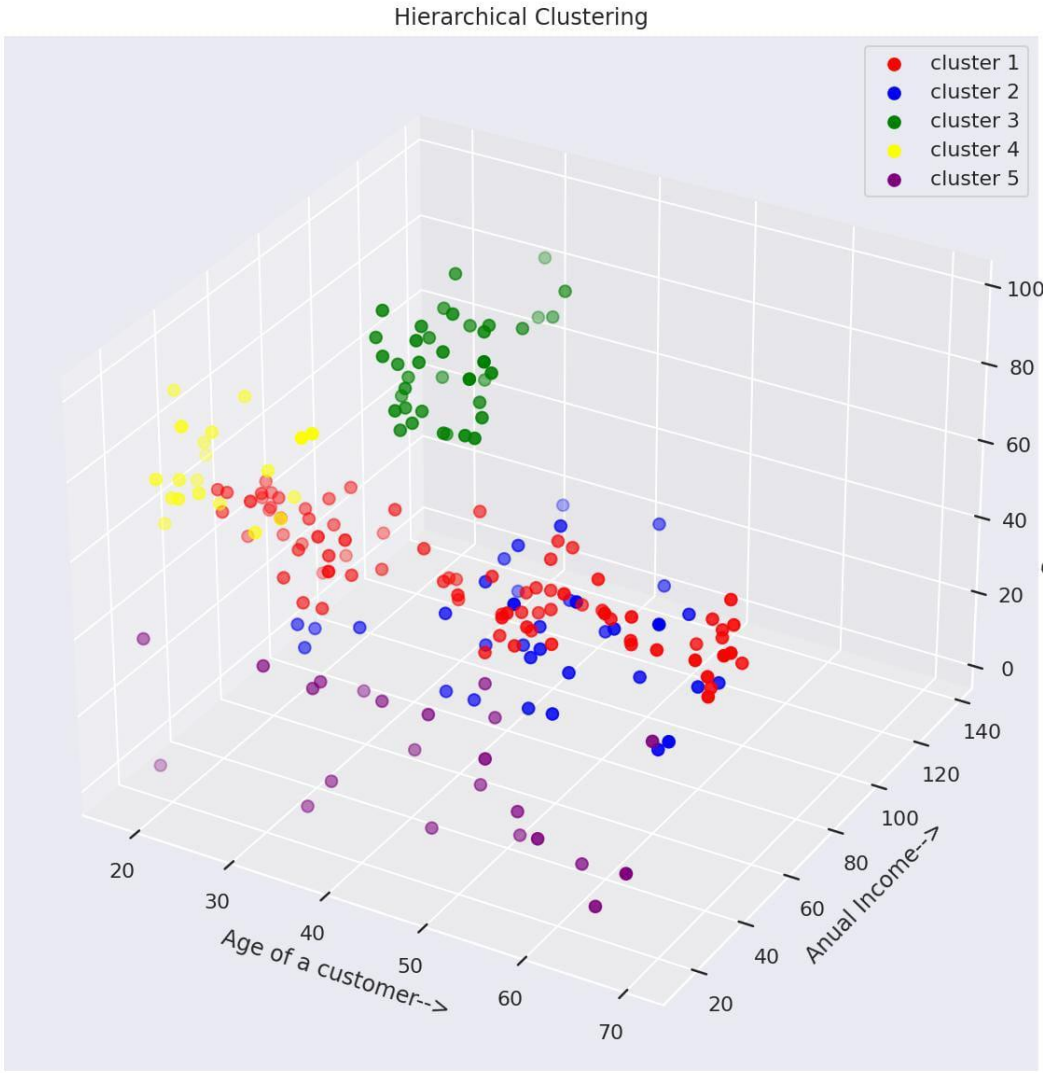
[308812.78,
212840.1698209719,
157170.21611721613,
105251.108132205,
79298.04768537768,
72407.70656816158,
70475.41923659378,
48883.32732076108,
40920.25435539312,
38601.21196679138]


Average Income and Spending by Segment
Annual_Income_kdollars
Spending_Score
Mean Value
0
20
40
60
80
HI-LS
MI-MS
HI-HS
LI-HS
LI-LS
Segment


WCSS value
300000
250000
200000
150000
100000
50000
2
4
6
8
10
Number of clusters


cluster 1
cluster 2
cluster 3
Spending Score-->
Anual Income-->
Age of a customer-->


Dendrogram
Euclidean Distance
Customers


Hierarchical Clustering
cluster 1
cluster 2
cluster 3
cluster 4
cluster 5
Age of a customer-->
Anual Income-->
Spending Score-->